Mini Courier Planner

Capstone Report

Student: Tanishq

Prototype: sorting + 0/1 knapsack DP + Dijkstra + nearest-neighbor TSP

GitHub: https://github.com/Tanishq4176/DAA-Capestone

Abstract

This project is an integrated courier-planning prototype. It reads parcel and road CSVs, ranks parcels, chooses a maximum-benefit subset that fits one vehicle, then builds a delivery tour that starts and ends at the warehouse. It also times the main algorithms on growing synthetic inputs and stores a runtime chart plus console traces. The design matches the five capstone phases: problem design, packing, routing, performance, and demonstration.

1. Problem and Design (Phase 1)

A courier receives parcels, each with destination, weight, declared value, deadline, and priority. One vehicle has a fixed weight capacity. The system must:

- Order parcels so high priority and tight deadlines are visible.
- Choose a subset that maximizes a transparent benefit score without exceeding capacity.
- Route the chosen destinations over a weighted road network and return to the warehouse.

Parcel attributes: parcel_id, destination, weight (kg), value, deadline (smaller = more urgent), priority in {High, Medium, Low}.

Benefit score: value + 20 x priority_score + 5 x max(0, 6 - deadline), with High/Medium/Low = 3/2/1. This is a planning proxy, not profit.

Graph format: undirected adjacency map from datasets/routes.csv. Each row is source, destination, distance with non-negative kilometres. The loader inserts both directions. Nodes are delivery points plus Warehouse.

Why these algorithms were selected:

Need	Algorithm	Suitability
Two-key ranking, keep ties	Python Timsort	Stable, O(n log n)
Best subset under weight	0/1 knapsack DP	Exact for integer kg
Shortest non-negative roads	Dijkstra + heap	Correct for km weights
Order stops and return	Nearest-neighbor	Fast TSP approximation

Flow: CSV validation -> stable sort -> 0/1 knapsack -> Dijkstra shortest paths -> nearest-neighbor tour -> report.

2. Package Selection (Phase 2)

The small dataset has nine parcels and a 12 kg vehicle. Sorting uses (-priority_score, deadline). Python sorted() is Timsort and is stable, so P04 stays before P09 when both are High with deadline 3.

Knapsack recurrence: dp[i][c] = max(dp[i-1][c], benefit[i] + dp[i-1][c-weight[i]]) when the parcel fits; otherwise dp[i][c] = dp[i-1][c]. The table is (n+1) x (capacity+1). Time and space are O(nC).

Small-instance result (capacity 12): selected P02, P01, P04, P09, P06, weight 12/12 kg, benefit 550. A

capacity-3 edge case selects a feasible subset with benefit 185.

```
Phase 2 DP table evidence (capacity 12 kg)
```

```
0/1 knapsack DP table (rows = parcels considered, columns = remaining kg):
        cap->    0    1    2    3    4    5    6    7    8    9   10   11   12
  (none)         0    0    0    0    0    0    0    0    0    0    0    0    0
  P02            0    0  120  120  120  120  120  120  120  120  120  120  120
  P01            0    0  120  120  140  140  260  260  260  260  260  260  260
  P04            0    0  120  120  140  240  260  260  260  380  380  380  380
  P09            0    0  120  120  225  240  260  345  365  380  380  485  485
  P07            0    0  120  120  225  240  260  345  365  380  380  485  485
  P05            0    0  120  120  225  240  260  345  365  380  380  485  485
  P03            0    0  120  120  225  240  260  345  365  380  380  485  485
  P06            0   65  120  185  225  290  305  345  410  430  445  485  550
  P08            0   65  120  185  225  290  305  350  410  430  470  490  550
```

Figure 1. 0/1 knapsack DP table for the 12 kg sample.

3. Graph and Route (Phase 3)

The sample graph has 10 nodes (Warehouse, A-I) and 19 undirected roads. Dijkstra from Warehouse on the selected stops: B 2 km, A 3 km, D 6 km, I 9 km, F 8 km.

Nearest-neighbor stop order: Warehouse -> B -> A -> D -> F -> I -> Warehouse. Total distance 23 km.

Heap Dijkstra is O((V+E) log V). Repeated Dijkstra for k stops is about O(k(V+E) log V). The tour is a heuristic, not an exact TSP solver.

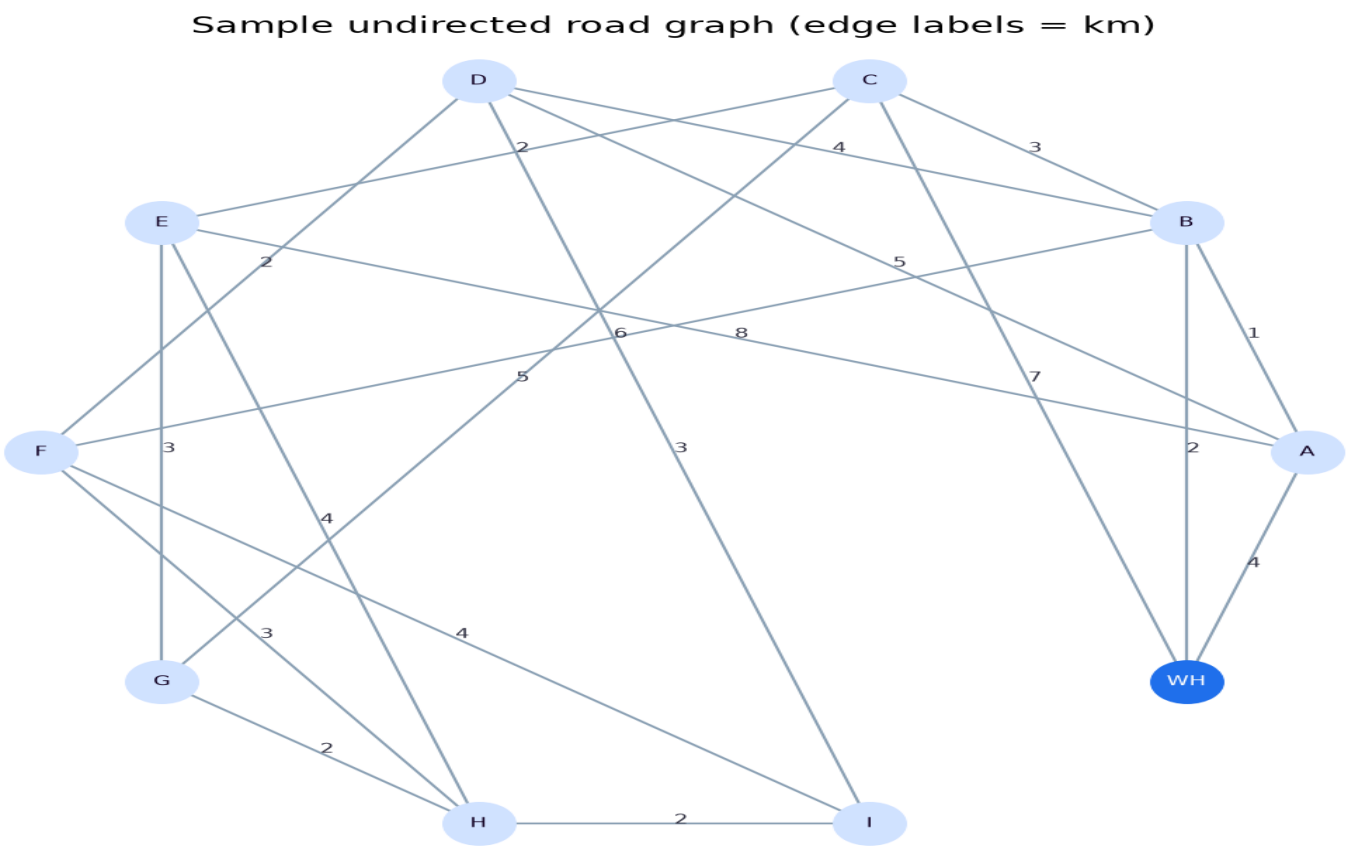


Figure 2. Sample undirected road graph (edge labels in km).

4. Performance Testing (Phase 4)

Two evidence layers were used. Real CSVs: 9 parcels / 10 nodes and 30 parcels / 17 nodes (capacity 25). Synthetic scaling: sizes 10, 50, 100, and 200, five repeats, median milliseconds.

Size	Capacity	Sort (ms)	Knapsack (ms)	Dijkstra (ms)	Route (ms)
10	12	0.0015	0.0146	0.0086	0.0815
50	25	0.0061	0.1404	0.0755	1.7294
100	50	0.0260	0.4821	0.1418	3.1458
200	100	0.0673	2.4628	0.2699	5.7993

These numbers are one Python/runtime/machine snapshot. Knapsack grows with both n and C. Route cost dominates. Future optimization: 1D DP storage, cached k x k distances, two-opt on the tour, and more benchmark repeats.

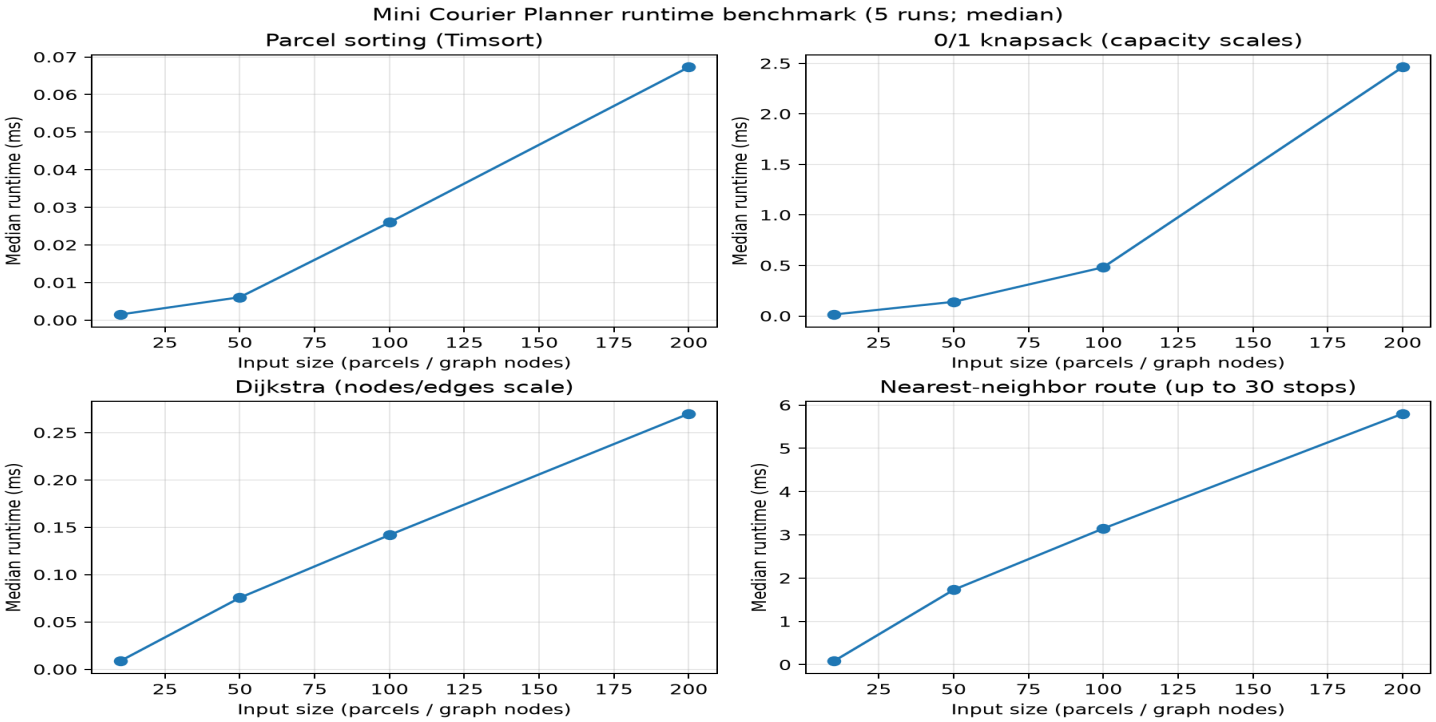


Figure 3. Median runtime of sorting, knapsack, Dijkstra, and routing.

5. Results and Demonstration (Phase 5)

Run py app.py for the 12 kg sample. The prototype validates IDs, fields, graph nodes, distances, capacity, and reachability, then prints the adjacency list, ranked parcels, DP table, packing plan, shortest paths, stop order, expanded road path, and total km.

It is a teaching demo, not a production dispatcher: one vehicle, no time windows, scalar benefit, heuristic tour.

Phase 5 demonstration — py app.py

```
MINI COURIER PLANNER
Student: Tanishq
Integrated prototype: sorting + 0/1 knapsack DP + Dijkstra + nearest-neighbor TSP
Vehicle capacity: 12 kg
Parcels file: parcels.csv
Routes file: routes.csv

Road graph (undirected adjacency list, distance in km):
  A: B:1, D:5, E:8, Warehouse:4
  B: A:1, C:3, D:4, F:6, Warehouse:2
  C: B:3, E:2, G:5, Warehouse:7
  D: A:5, B:4, F:2, I:3
  E: A:8, C:2, G:3, H:4
  F: B:6, D:2, H:3, I:4
  G: C:5, E:3, H:2
  H: E:4, F:3, G:2, I:2
  I: D:3, F:4, H:2
  Warehouse: A:4, B:2, C:7

Phase 2 sorting: Python Timsort, key = (-priority_score, deadline).
Timsort is stable, so equal priority+deadline parcels keep input order.
Prioritized parcels (High to Low, then earliest deadline):
  P02 -> B | High   | deadline 1 | 2 kg | benefit 120
  P01 -> A | High   | deadline 2 | 4 kg | benefit 140
  P04 -> D | High   | deadline 3 | 3 kg | benefit 120
  P09 -> I | High   | deadline 3 | 2 kg | benefit 105
  P07 -> G | High   | deadline 5 | 7 kg | benefit 165
  P05 -> E | Medium | deadline 2 | 5 kg | benefit 130
  P03 -> C | Medium | deadline 4 | 6 kg | benefit 130
  P06 -> F | Low    | deadline 1 | 1 kg | benefit 65
  P08 -> H | Low    | deadline 3 | 2 kg | benefit 60

Packing plan: P02, P01, P04, P09, P06
Total weight: 12/12 kg
Optimal benefit: 550
Unused capacity: 0 kg
DP table size: 10 rows x 13 columns (O(nC) time and space)

0/1 knapsack DP table (rows = parcels considered, columns = remaining kg):
        cap->    0    1    2    3    4    5    6    7    8    9   10   11   12
  (none)         0    0    0    0    0    0    0    0    0    0    0    0    0
  P02            0    0  120  120  120  120  120  120  120  120  120  120  120
  P01            0    0  120  120  140  140  260  260  260  260  260  260  260
  P04            0    0  120  120  140  240  260  260  260  380  380  380  380
  P09            0    0  120  120  225  240  260  345  365  380  380  485  485
  P07            0    0  120  120  225  240  260  345  365  380  380  485  485
  P05            0    0  120  120  225  240  260  345  365  380  380  485  485
  P03            0    0  120  120  225  240  260  345  365  380  380  485  485
  P06            0   65  120  185  225  290  305  345  410  430  445  485  550
  P08            0   65  120  185  225  290  305  350  410  430  470  490  550

Shortest paths from warehouse (Dijkstra, non-negative km):
  Warehouse -> B | 2 km
  Warehouse -> B -> A | 3 km
  Warehouse -> B -> D | 6 km
  Warehouse -> B -> D -> I | 9 km
  Warehouse -> B -> F | 8 km

Nearest-neighbor stop order: Warehouse -> B -> A -> D -> F -> I -> Warehouse
Expanded road path: Warehouse -> B -> A -> D -> F -> I -> D -> B -> Warehouse
Total route distance: 23 km

Note: nearest-neighbor is a TSP approximation. It is not guaranteed to be a globally shortest tour.
```

Figure 4. Console demonstration of py app.py (small instance).

Conclusion and future scope

The project joins Timsort, 0/1 knapsack, Dijkstra, and a TSP approximation in one reproducible workflow. Packing is exact for the chosen score and capacity. Routing is practical and approximate. Future work: time windows, travel times, exact TSP on tiny stop sets, multi-vehicle models, and larger real maps.